TECHLEARN ACADEMY

Orientação a objetos, MVC e padrões de projeto

Curso

Programação Orientada a Objetos e UML

Módulo 4 - Reutilização, MVC e padrões de projeto

Como estudar esta matéria

1. Leia a explicação teórica e acompanhe os exemplos do material.

2. Resolva os exercícios comentados antes de conferir suas justificativas. Anote suas dúvidas para conversar com o mentor.

3. Ao terminar os materiais do módulo, faça a avaliação de revisão disponível no site.

Origem do conteúdo

CQA/UNIP - Tecnologia em Análise e Desenvolvimento de Sistemas, Tomo 10. Páginas 24 a 34 do arquivo original.

O conteúdo da matéria começa na página 4 deste PDF. Antes dele, foram mantidas a capa e a página de créditos e avisos do documento original.

Os textos e exercícios originais conservam suas referências e o contexto de publicação. A organização em cursos e as avaliações de revisão pertencem ao projeto TechLearn Academy.

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

MATERIAL INSTRUCIONAL ESPECÍFICO

TOMO 10

CQA/UNIP – Comissão de Qualificação e Avaliação da UNIP

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

MATERIAL INSTRUCIONAL ESPECÍFICO

TOMO 10

Christiane Mazur Doi

Doutora em Engenharia Metalúrgica e de Materiais, Mestra em Ciências - Tecnologia Nuclear, Especialista em Língua Portuguesa e Literatura, Engenheira Química e Licenciada em Matemática, com Aperfeiçoamento em Estatística. Professora titular da Universidade Paulista.

Tiago Guglielmeti Correale

Doutor em Engenharia Elétrica, Mestre em Engenharia Elétrica e Engenheiro Elétrico (ênfase em Telecomunicações). Professor titular da Universidade Paulista.

Material instrucional específico, cujo conteúdo integral ou parcial não pode ser reproduzido ou utilizado sem autorização expressa, por escrito, da CQA/UNIP – Comissão de Qualificação e Avaliação da UNIP – UNIVERSIDADE PAULISTA.

Questões 7, 8 e 9

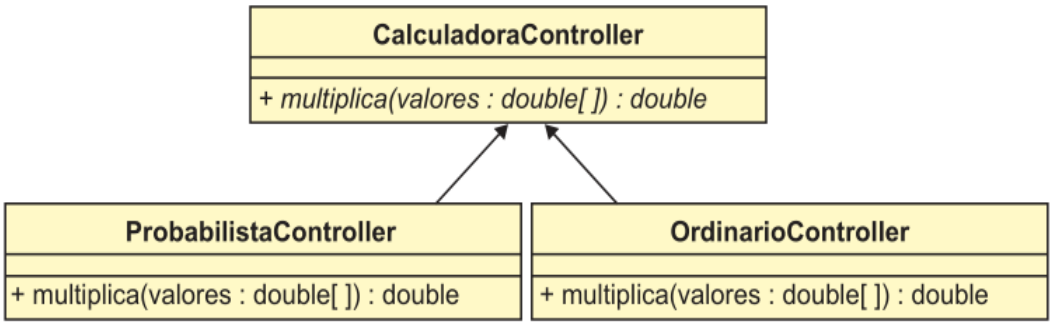
Questão 7.[7]

Para que se realize a multiplicação de probabilidades com maior facilidade, foi desenvolvida, utilizando-se o padrão MVC (*Model-View-Controller*), uma calculadora que pode ser configurada de dois modos: comum e probabilista. No primeiro, as multiplicações ocorrem de forma ordinária. No segundo, as multiplicações são feitas utilizando-se uma fórmula específica. A classe a seguir, escrita na linguagem Java, define o método `multiplica`. Ele é executado quando o botão da multiplicação é pressionado, e recebe como parâmetro os valores que o usuário deseja usar na operação, além de possuir uma variável de instância do tipo `CalculadoraController`. O método `alteraModo` é encarregado de alterar a instância para a qual essa variável faz referência conforme o modo selecionado pelo usuário. A instância interage com as partes do *model* apropriadas à solicitação realizada pelo usuário. Quando o usuário deseja usar o modo probabilista, a variável `modo` tem o valor 1, caso contrário, ela tem o valor 0.

```
public class Calculadora {
      private CalculadoraController c;
      private int modo;
      public double multiplica (double[] valores) {
            return c.multiplica (valores);
      }
      public void alteraModo () {
            if ( modo == 1)
                  c = new ProbabilistaController();
            else
            c = new OrdinarioController();
      }
}
```

O diagrama de classe que se segue mostra a hierarquia de *controllers* definida para a implementação da calculadora descrita. A hierarquia de classes de controle representa uma família de algoritmos intercambiáveis, por isso, o comportamento dos componentes da *view*, a cada instante, pode ser dinamicamente alterado, bastando trocar o tipo da instância referenciada pela variável `c`.

[7]Questão 26 – Enade 2017.



Considerando esse cenário, avalie as afirmativas.

I. A hierarquia exibida ilustra o uso do padrão de projetos *Strategy*.

II. O padrão composto MVC define a existência de um único *controller* e, portanto, a solução proposta não caracteriza o uso desse padrão.

III. O método `multiplica` da classe `Calculadora` chama o método `multiplica` de `CalculadoraController`, o que caracteriza uma conversão de interfaces e, portanto, o uso do padrão de projetos *Adapter*.

É correto o que se afirma em

A. I, apenas.

B. II, apenas.

C. I e III, apenas.

D. II e III, apenas.

E. I, II e III.

Questão 8.[8]

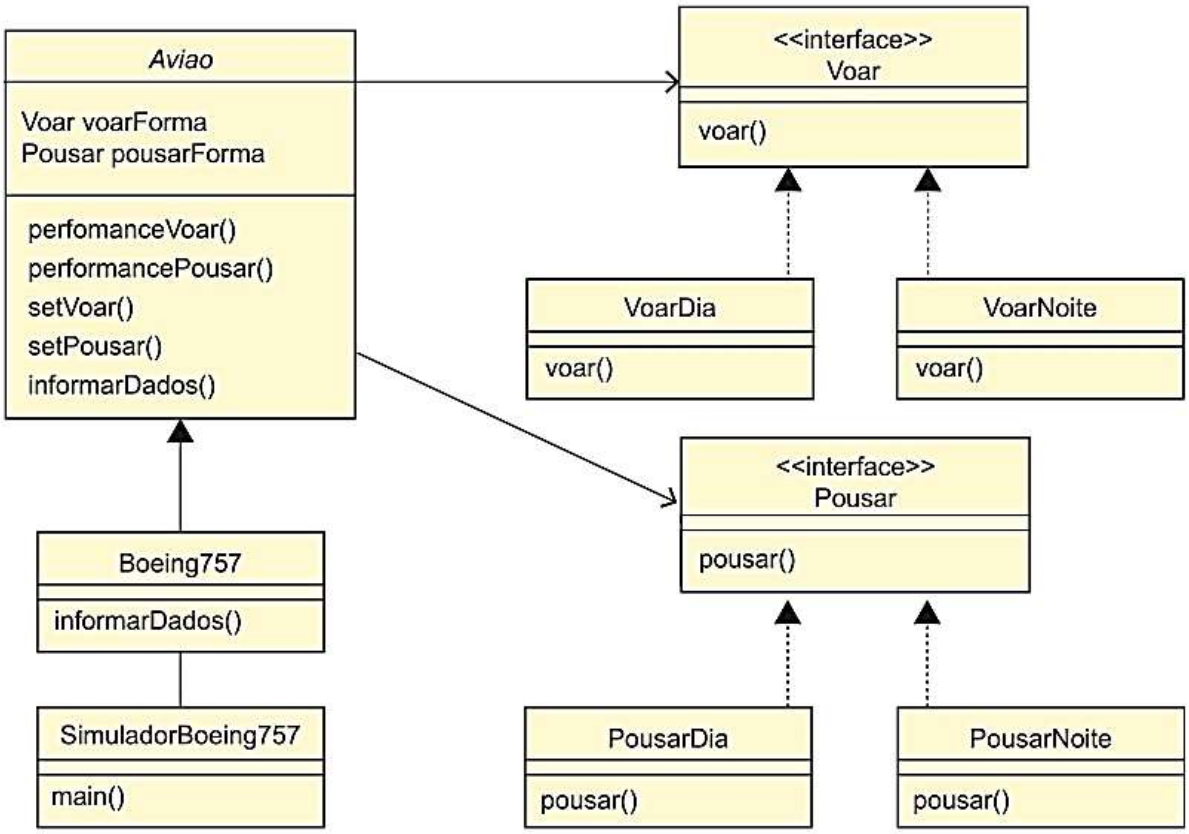
Leia o texto a seguir.

O diagrama de classe apresentado na figura a seguir mapeia um processo de um simulador de voo. Nesse diagrama, é utilizado o padrão de projeto Strategy, que define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. O padrão Strategy, que deixa o algoritmo variar independentemente dos clientes que o utilizam, aplica o princípio de projeto: "programar para uma interface, não para uma implementação".

LARMAN, C. *Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo*. Porto Alegre: Bookman, 2007 (com adaptações).

Considere o diagrama de classe a seguir que descreve um simulador de voo.

[8]Questão 27 – Enade 2017.



Considere, ainda, o seguinte código em linguagem Java como implementação do simulador.

```
public interface Voar {
     public void voar();
}
public interface Pousar {
     public void pousar();
}
public class VoarDia implements Voar {
     public void voar() {
          System.out.println(“Voar durante o dia.”);
     }
}
public class VoarNoite implements Voar {
     public void voar() {
          System.out.println(“Voar durante a noite.”);
     }
}
public class PousarDia implements Pousar {
     public void pousar() {
          System.out.println(“Pousar durante o dia.”);
     }
}

public class PousarNoite implements Pousar {
     public void pousar() {
```

```
            System.out.println("Pousar durante a noite.");
      }
}
public abstract class Aviao {
      Voar voarForma;
      Pousar pousarForma;
      public Aviao() {}
      public abstract void informarDados();
      public void performanceVoar() {
                  voarForma.voar();
      }
      public void performancePousar() {
            pousarForma.pousar();
      }
      public void setVoar(Voar v) {
            voarForma = v;
      }
      public void setPousar(Pousar p) {
            pousarForma = p;
      }
}
public class Boeing757 extends Aviao {
      public Boeing757() {
            voarForma = new VoarDia();
            pousarForma = new PousarDia();
      }

      public void informarDados() {
            System.out.println("Informando dados de um Boeing 757.");
      }
}
public class SimuladorBoeing757 {
      public static void main(String[] args) {
            Boeing757 b757 = new Boeing757();
            b757.performanceVoar();
            b757.performancePousar();
            b757.setVoar(new VoarNoite());
            b757.setPousar(new PousarNoite());
            b757.performanceVoar();
            b757.performancePousar();
      }
}
```

Com base no diagrama de classe, no código em linguagem Java e no conceito do padrão de projeto *Strategy*, assinale a opção de correta.

A. A classe `Aviao` implementa as interfaces `Voar` e `Pousar`.

B. A classe `Boeing757` não aplica o princípio de herança, uma vez que esse princípio já foi aplicado pela classe `Aviao`, nas interfaces `Voar` e `Pousar`.

C. Por implementarem as interfaces `Voar` e `Pousar`, as classes `VoarDia`, `VoarNoite`, `PousarDia` e `PousarNoite` também são uma interface.

D. Na classe `SimuladorBoeing757`, os métodos `setVoar` e `setPousar` são os responsáveis por aplicar, respectivamente, as saídas "Voar durante o dia" e "Pousar durante o dia" para as saídas "Voar durante a noite" e "Pousar durante a noite".

E. Quando for executada, a classe `SimuladorBoeing757` apresentará um erro em tempo de execução porque os métodos `setVoar` e `setPousar` recebem respectivamente, como parâmetros, variáveis do tipo classe, o que não é permitido na orientação a objetos.

Questão 9.[9]

A área de desenvolvimento de *software* está se tornando cada vez mais complexa. Para lidar com essa realidade, os desenvolvedores contam com linguagens de programação baseadas no paradigma de orientação a objetos, cujos pilares são abstração, encapsulamento, herança e polimorfismo. No código a seguir, observa-se a implementação de classes relacionadas.

```
public abstract class Impressora {
      String nome;
      Impressora() {}
      Impressora(String n) {
            this.nome = n;
      }
      public void imprimir() {}
}
public class Laser extends Impressora {
      public Laser() {}
      public void imprimir() {
            System.out.println(“Imprimindo na Laser”);
      }
}
class Matricial extends Impressora {
      public Matricial () {}
      public void imprimir() {
            System.out.println(“Imprimindo na Matricial”);
      }
}
public class JatoDeTinta extends Impressora {
      public JatoDeTinta() {}
      public void imprimir(){
            System.out.println(“Imprimindo na Jato de tinta”);
      }
}
```

[9]Questão 30 – Enade 2017.

```
public class Main {
      public static void main(String args[]) {
            Impressora imp[] = new Impressora[3];
            imp[0] = new Laser();
            imp[1] = new JatoDeTinta();
            imp[2] = new Matricial();
            for(int i = imp.length - 1; i >= 0; i--){
                  imp[i].imprimir();
            }
      }
}
```

Com base nas informações do texto e no código apresentado, avalie as afirmativas a seguir.

I. A execução do código, via classe `Main`, resulta na seguinte saída:

Imprimindo na Laser

Imprimindo na Matricial

Imprimindo na Jato de tinta

II. O código faz uso da técnica denominada polimorfismo.

III. O código não será compilado, pois o vetor `imp` foi instanciado por meio da classe abstrata `Impressora`.

É correto o que se afirma em

A. II, apenas.

B. III, apenas.

C. I e II, apenas.

D. I e III, apenas.

E. I, II e III.

1. Análise da questão

Orientação a objetos e padrões de projeto

As três questões apresentadas exploram conceitos fundamentais de orientação a objetos e de padrões de projeto.

Existem diferentes abordagens no que se refere à orientação a objetos, mas Booch et al. (2007) afirmam que "*o conceito fundamental é que se deve modelar um sistema de software como uma coleção de objetos, sendo estes instâncias de uma classe dentro de uma hierarquia de classes".*

De forma similar, outros autores, podem focar mais no aspecto de projeto: "o *projeto orientado a objetos decompõe um sistema em objetos, sendo esses seus componentes básicos"* (WIRFS-BROCK, WILKERSON e WIENER, 1990). Nesse mesmo texto, os autores focam nos mecanismos de abstração fornecidos pela orientação a objetos.

Page-Jones (2001) destaca que nove conceitos são fundamentais para a orientação a objetos, listados a seguir.

- Encapsulamento.
- Ocultação de informações e implementações.
- Retenção de estado.
- Identidade de objeto.
- Mensagens.
- Classes.
- Herança.
- Polimorfismo.
- Generalização.

Apenas o uso de uma linguagem orientada a objetos não garante a escrita de um software de qualidade (LARMAN, 2007). Projetistas experientes de software observaram que muitos problemas são recorrentes, ainda que ocorram variações, e podem ser resolvidos por estratégias similares. Além disso, algumas dessas estratégias parecem ser melhores do que outras: enquanto determinadas soluções podem apresentar problemas de manutenção no longo prazo, outras podem ser mais flexíveis e eficazes.

As melhores soluções deram origem aos chamados "padrões de projeto". Problemas similares foram catalogados, juntamente com as melhores estratégias para as suas soluções. Muitas dessas estratégias foram desenvolvidas com o objetivo de utilizar as ideias da orientação a objetos da melhor forma possível.

Em alguns livros, como na publicação de Larman (2007), busca-se ensinar simultaneamente conceitos de orientação a objetos, UML e padrões de projeto. Outros livros, como a publicação de Gamma et al. (1995), têm uma abordagem mais diretamente focada nos padrões de projeto em si e funcionam como uma espécie de catálogo.

O padrão *Model-view-controller* (MVC) foi, de modo inicial, realizado na linguagem Smalltalk-80, justamente para o desenvolvimento de interfaces gráficas (LARMAN, 2007; GAMMA et al., 1995). A ideia básica é isolar três tipos de objetos:

- objetos relacionados à apresentação ou à interface gráfica, que correspondem ao tipo *View*;

- objetos relacionados à forma como a interface gráfica reage às ações do usuário, que correspondem ao tipo *Control*;
- objetos relacionados ao modelo da aplicação propriamente dita, que correspondem ao tipo *Model.*

Larman (2007) adiciona que o padrão MVC é uma aplicação de um princípio mais básico: o princípio da separação entre o modelo (relacionado ao domínio) e a visualização (relacionada à forma de apresentação). Esse conceito foi expandido ao longo do tempo e deu origem a um padrão arquitetural utilizado de maneira mais sofisticada em sistemas distribuídos, especialmente em sistemas *web*.

Gamma et al. (1995), no livro clássico sobre padrões de projeto, estabelecem quatro elementos essenciais da definição de um padrão de projeto:

- o nome do projeto (para referência);
- a descrição do problema (ou das classes de problemas) ao qual o padrão se aplica;
- o estabelecimento da estrutura da solução;
- a determinação das consequências da aplicação do padrão, ou seja, dos compromissos que fazemos ao adotarmos dado padrão.

Com relação à solução, devemos ter mente que ela não é uma descrição detalhada e concreta para um problema em específico, mas uma ideia abstrata de como resolver uma classe grande de problemas (GAMMA et al., 1995). Assim, as soluções apresentadas pelos padrões de projeto não focam em uma implementação específica, mas, sim, em guias gerais para a construção dessas soluções, propostas por projetistas ou programadores.

Um exemplo de padrão é o *Strategy* (estratégia). Esse padrão, também chamado de *policy* (política), é utilizado no caso em que existem diferentes algoritmos (ou políticas) que devem ser aplicados em diferentes contextos. A solução busca encapsular cada família de algoritmos em diferentes classes e estabelecer uma interface de acesso para eles.

2. Análise das afirmativas e das alternativas

Questão 7.

I – Afirmativa correta.

JUSTIFICATIVA. A ideia do padrão de projetos *Strategy* é criar uma interface para acessar diferentes tipos de algoritmos. No caso do problema apresentado, isso envolve uma classe

para multiplicações comuns (chamada de ordinária no enunciado) e probabilistas, além da definição de uma interface única de acesso a essas classes.

II – Afirmativa incorreta.
JUSTIFICATIVA. É possível utilizar mais de um *controller* no padrão MVC.

III – Afirmativa incorreta.
JUSTIFICATIVA. Não ocorre conversão de interfaces nem é utilizado o padrão *Adapter*.

Alternativa correta: A.

Questão 8.

A – Alternativa incorreta.
JUSTIFICATIVA. A classe Aviao é uma classe abstrata e não implementa as interfaces Voar e Pousar. O tipo de relação entre Aviao e Voar e entre Aviao e Pousar é chamado de associação.

B – Alternativa incorreta.
JUSTIFICATIVA. A classe Boeing757 utiliza o princípio de herança, o que pode ser visto tanto pelo diagrama de classes mostrado no enunciado quanto no código-fonte, uma vez que é utilizado a *keyword extends* na definição da classe Boeing757.

C – Alternativa incorreta.
JUSTIFICATIVA. Justamente por implementarem uma interface, essas classes não são uma interface.

D – Alternativa correta.
JUSTIFICATIVA. Os métodos setVoar e setPousar recebem objetos cujas classes implementam as interfaces de Voar e Pousar. A classe Boeing757, no seu construtor, inicializa as referências voarForma e pousarForma com objetos das classes VoarDia e PousarDia. Contudo, a classe SimuladorBoeing757 altera essas referências para objetos das classes VoarNoite e PousarNoite pelos métodos setVoar e setPousar. Esse comportamento é descrito na alternativa.

E – Alternativa correta.

JUSTIFICATIVA. Não existe um erro, pois esse é o mecanismo básico de passagem por referência da linguagem Java.

Questão 9.

I – Afirmativa incorreta.

JUSTIFICATIVA. A saída do programa será a que segue.

Imprimindo na Matricial
Imprimindo na Jato de tinta
Imprimindo na Laser

Devemos observar que o laço de impressão começa da última posição do vetor (que corresponde a Matricial), indo até a primeira posição (que corresponde a Laser).

II – Afirmativa correta.

JUSTIFICATIVA. Podemos verificar que o método imprimir() é chamado para os objetos contidos no vetor imp. Observe que a classe Impressora é uma classe abstrata, e, mesmo que o método imprimir não tenha sido declarado como abstrato, ele está vazio. Isso não é exatamente a mesma coisa, mas, no contexto do enunciado, fica evidente que as classes que herdam a classe Impressora terão de implementar esses métodos (que é, de fato, o que ocorre). Assim, podemos dizer que o programa do enunciado utiliza polimorfismo.

III – Afirmativa incorreta.

JUSTIFICATIVA. O vetor imp é um vetor de referências para as diversas impressoras. Assim, cada elemento desse vetor deve conter um objeto que apresenta uma implementação do método imprimir(), mas o vetor em si é um vetor de referências.

Alternativa correta: A.

2. Indicações bibliográficas

- BOOCH, G. et al. *Object-oriented analysis and design with applications.* 3. ed. Boston: Pearson Prentice Hall, 2007.
- GAMMA, E. et al. *Design patterns: elements of reusable object-oriented software.* Boston: Addison-Wesley, 1995.
- LARMAN, C. *Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo*. Porto Alegre: Bookman, 2007.
- PAGE-JONES, M. *Fundamentos do desenho orientado a objeto com UML.* São Paulo: Pearson, 2001.
- WIRFS-BROCK, R.; WILKERSON, B.; WIENER, L. *Designing object-oriented software.* Englewoods Cliffs: Prentice-Hall, 1990.